

Лабораторна робота №1

Асинхронна обробка даних у TypeScript

3. Завдання

3.1. Встановлення Node.js

Встановити Node.js на робочий комп'ютер.

Вимоги:

- Завантажити LTS-версію з <https://nodejs.org>.
- Переконаватися, що команди `node --version` та `npm --version` працюють у терміналі.

3.2. Налаштування проєкту з TypeScript

Створити новий проєкт та налаштувати TypeScript.

Вимоги:

- Ініціалізувати проєкт: `npm init -y`.
- Встановити TypeScript: `npm install -D typescript`.
- Створити `tsconfig.json` через `npx tsc --init`.
- Створити папку `src/` для вихідного коду.
- Додати скрипт "build": "tsc" у `package.json`.
- Переконаватися, що `npm run build` компілює файли без помилок.

3.3. Функція `delay`

Сигнатура:

```
delay(ms: number): Promise<void>
```

Створює Promise, що завершується через вказаний час. Використовується як допоміжна функція в інших завданнях.

Вимоги:

- Приймає кількість мілісекунд затримки.
- Повертає `Promise<void>`, що завершується через `ms` мілісекунд.
- Використати конструктор `new Promise` та `setTimeout`.

Приклад використання:

```
await delay(1000); // чекаємо 1 секунду
console.log('Готово');
```

Лістинг програми:

```
export const delay = (ms: number): Promise<void> => {
  return new Promise((resolve) => setTimeout(resolve, ms + 15));
};
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.21.121.23.000 – Лр.1		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Сорока А.О.			Звіт з лабораторної роботи №1	Літ.	Арк.
Перевір.		Фуріхата Д. В.					1
Реценз.						ФІКТ, гр. ВТ-23-1	
Н. Контр.							
Зав.каф.							

3.4. Функція `fetchUserProfiles`

Сигнатура:

```
fetchUserProfiles(userIds: string[]): Promise<UserProfile[]>
```

Інтерфейс:

```
interface UserProfile {  
  id: string;  
  name: string;  
  email: string;  
}
```

Симулює паралельне завантаження профілів користувачів з сервера.

Вимоги:

- Кожен запит має симулювати затримку 50–150 мс (випадково).
- Використати `Promise.all` для паралельного завантаження.
- Якщо `userIds` порожній — повернути порожній масив.
- Всі змінні та параметри мають бути типізовані.

Приклад використання:

```
const profiles = await fetchUserProfiles(['1', '2', '3']);  
// Паралельне завантаження ~100мс, не ~300мс  
console.log(profiles);  
// [  
//   { id: '1', name: 'User 1', email: 'user1@example.com' },  
//   { id: '2', name: 'User 2', email: 'user2@example.com' },  
//   { id: '3', name: 'User 3', email: 'user3@example.com' }  
// ]
```

Лістинг програми:

```
export async function fetchUserProfiles(userIds: string[]): Promise<UserProfile[]>  
{  
  if (userIds.length === 0) return [];  
  
  const promises = userIds.map(async (id) => {  
    const randomDelay = Math.floor(Math.random() * (1500 - 50 + 1)) + 50;  
    await delay(randomDelay);  
    return {  
      id,  
      name: `User ${id}`,  
      email: `user_${id}@test.ua`  
    };  
  });  
  
  return Promise.all(promises);  
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.1	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

3.5. Функція retryOperation

Сигнатура:

```
retryOperation(  
  operation: () => Promise<unknown>,  
  maxRetries: number  
): Promise<unknown>
```

Виконує асинхронну операцію з автоматичними повторними спробами при помилці.

Вимоги:

- При помилці — зачекати 100 мс, спробувати знову.
- Максимум maxRetries спроб (за замовчуванням 3).
- Якщо всі спроби провалилися — викинути останню помилку.
- Логувати кожну спробу: `console.log('Спроба ${n}...')`.

Приклад використання:

```
let attempts = 0;  
const unreliableOperation = async (): Promise<string> => {  
  attempts++;  
  if (attempts < 3) {  
    throw new Error('Тимчасова помилка');  
  }  
  return 'Успіх!';  
};  
  
const result = await retryOperation(unreliableOperation, 3);  
// Лог: Спроба 1... → помилка → чекаємо 100мс  
//      Спроба 2... → помилка → чекаємо 100мс  
//      Спроба 3... → успіх!  
console.log(result); // 'Успіх!'
```

Лістинг програми:

```
export async function retryOperation<T>(  
  operation: () => Promise<T>,  
  maxRetries: number = 3  
) : Promise<T> {  
  let lastError: any;  
  
  for (let i = 1; i <= maxRetries; i++) {  
    try {  
      console.log(`Спроба ${i}...`);  
      return await operation();  
    } catch (error) {  
      lastError = error;  
      if (i < maxRetries) {  
        await delay(150);  
      }  
    }  
  }  
  throw lastError;  
}
```

3.6. Функція processInBatches

Сигнатура:

```
processInBatches(  
  items: number[],  
  batchSize: number,  
  processor: (batch: number[]) => Promise<number[]>  
) : Promise<number[]>
```

Обробляє великий масив елементів партіями (batches) послідовно.

Вимоги:

- Розділити items на партії розміром batchSize.
- Обробити партії **послідовно** (не паралельно!).
- Кожну партію обробити функцією processor.
- Повернути масив усіх результатів.
- Логувати прогрес: console.log('Обробка партії \${n}/\${total}...').

Приклад використання:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];  
  
const processor = async (batch: number[]): Promise<number[]> => {  
  await delay(100); // Симуляція обробки  
  return batch.map(n => n * 2);  
};  
  
const results = await processInBatches(numbers, 3, processor);  
// Партія 1: [1,2,3] → [2,4,6] → чекаємо  
// Партія 2: [4,5,6] → [8,10,12] → чекаємо  
// Партія 3: [7,8] → [14,16] → готово  
console.log(results); // [2, 4, 6, 8, 10, 12, 14, 16]
```

Лістинг програми:

```
export async function processInBatches<T, R>(  
  items: T[],  
  batchSize: number,  
  processor: (batch: T[]) => Promise<R[]>  
) : Promise<R[]> {  
  const results: R[] = [];  
  const totalBatches = Math.ceil(items.length / batchSize);  
  
  for (let i = 0; i < items.length; i += batchSize) {  
    const batch = items.slice(i, i + batchSize);  
    const batchNumber = Math.floor(i / batchSize) + 1;  
    console.log(`Обробка партії ${batchNumber}/${totalBatches}...`);  
  
    if (batchNumber === 1) {  
      results.push(...(batch as unknown as R[]));  
    } else {  
      const processed = await processor(batch);  
      results.push(...processed);  
    }  
  }  
  return results;  
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.1	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		4

3.7. Функція `raceWithTimeout`

Сигнатура:

```
raceWithTimeout(  
  promise: Promise<unknown>,  
  timeoutMs: number  
): Promise<unknown>
```

Виконує Promise з обмеженням часу виконання.

Вимоги:

- Використати `Promise.race`.
- Якщо promise не завершився за `timeoutMs` — викинути помилку.
- Текст помилки: `'Operation timed out after ${timeoutMs}ms'`.

Приклад використання:

```
// Швидка операція – успіх  
const fast = delay(50).then(() => 'Готово');  
const result1 = await raceWithTimeout(fast, 100);  
console.log(result1); // 'Готово'  
  
// Повільна операція – timeout  
const slow = delay(200).then(() => 'Готово');  
try {  
  await raceWithTimeout(slow, 100);  
} catch (err) {  
  if (err instanceof Error) {  
    console.error(err.message); // 'Operation timed out after 100ms'  
  }  
}
```

Лістинг програми:

```
export async function raceWithTimeout<T>(  
  promise: Promise<T>,  
  timeoutMs: number  
) : Promise<T> {  
  const timeoutPromise = new Promise<never>((_ , reject) => {  
    setTimeout(() => {  
      const err = new Error(`Operation timed out after ${timeoutMs}ms`);  
      (err as any).timeoutMs = timeoutMs;  
      reject(err);  
    }, timeoutMs);  
  });  
  
  return Promise.race([promise, timeoutPromise]);  
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.1	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання всіх тестів:

```
Run start x
C:\Program Files\nodejs\npm.cmd run start

> node@1.0.0 start
> tsx src/main.ts

=== Тест 1: Паралельне завантаження ===
fetchUserProfiles: 1.457s
Завантажено 5 профілів

=== Тест 2: Retry механізм ===
Спроба 1...
Спроба 2...
Успіх!

=== Тест 3: Batch обробка ===
Обробка партії 1/4...
Обробка партії 2/4...
Обробка партії 3/4...
Обробка партії 4/4...
Оброблено: [
  1, 2, 3, 8, 10,
  12, 14, 16, 18, 20
]

=== Тест 4: Timeout ===
Timeout: Operation timed out after 100ms
```